



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ИНФОРМАЦИОННЫЕ СИСТЕМЫ В ХТ

Лекция 16. Веб-технологии

Содержание лекции

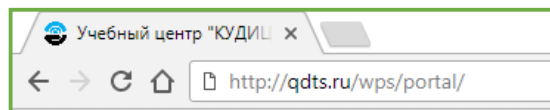
В данной лекции будут рассмотрены следующие темы:

-  Протокол HTTP
-  Технологии WWW
-  Язык разметки HTML
-  Каскадные стили CSS
-  Интерактивность веб-страниц: JavaScript

Интернет и всемирная паутина

- ⊙ Интернет - всемирная система объединённых компьютерных сетей для хранения и передачи информации
 - Интернет децентрализован
 - Каждый из миллионов компьютеров работает независимо
 - Более 3,800,000,000 пользователей в мире (51% от общей численности)
 - Основа – стек протоколов TCP/IP
 - Службы DNS, E-mail, WWW, FTP, Telnet ...
- ⊙ Всемирная паутина (World Wide Web, WWW) — распределённая модель доступа к связанным документам
 - Миллионы веб-серверов сети Интернет предоставляют доступ к документам
 - Документы размечены специальным образом и связаны между собой гиперссылками
 - Более 1,275,000,000 веб-сайтов

Принципиальная схема веб



1

Веб-сервер

Запрос HTTP

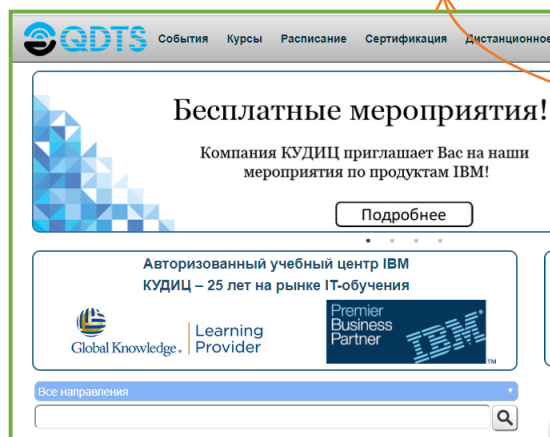
Браузер

3



Документ HTML

Ответ HTTP



```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1-
<html xmlns="http://www.w3.org/1999/xhtml"
<head>
<meta http-equiv="Content-Type" content="te
<!-- rel-dynamic-content indicates an eleme
dyn-cs:* URIs are resolved using the w
also be set using theme metadata if a
<link rel="stylesheet" href="/wps/contentha
offset=0&offset=10&themeID=23_905
type=text%2Fcss&entry=wp_preview_0.0%2
ead_css&entry=wp_theme_portal_80_0.0%2
head_css" type="text/css"/><link rel="style
offset=10&offset=14&themeID=23_905
type=text%2Fcss&entry=wp_preview_0.0%2
ead_css&entry=wp_theme_portal_80_0.0%2
head_css" type="text/css"/><link rel="alter
offset=0&offset=14&themeID=23_905
type=text%2Fcss&entry=wp_tagging_rating
p:entry=mm_builder_dialogs_0.0%3Ahead_css&
css&entry=wp_analytics_0.0%3Ahead_css&
type="text/javascript">var djConfig=
{"baseUrl":"/wps/portal_dojo/v1.7/dojo/", "1
{"com":"/wps/themeModules/js/com","ibm":"/w
type="text/javascript" src="/wps/contentha
themeID=23_9058104078V260INNNOOQ2006&amp;1
type=text%2Fplain&entry=wp_client_main_
1_80_0.0%3Ahead_js"></script><link rel="al
themeID=23_9058104078V260INNNOOQ2006&amp;1
type=text%2Fplain&entry=wp_client_main_1_80_0.0%2
```

2

Браузер

- Браузер — прикладное программное обеспечение для просмотра веб-страниц
 - Запрос ресурса по URL <http://en.wikipedia.org/>
 - Разбор документа (парсинг HTML, XML)
 - Отображение документа на основе стилей и разметки документа
 - Загрузка изображений, медиа файлов и пр.
 - Загрузка скриптов и библиотек
 - Выполнение скриптов JavaScript



Универсальный идентификатор ресурса

- URI (Uniform Resource Identifier) – строка, позволяющая идентифицировать ресурс
- URL (Uniform Resource Locator) – подвид URI, позволяющий определить расположение ресурса

схема : // логин : пароль @ хост : порт / путь ? параметры # якорь

- Схема, сетевой протокол – **http**, **https**, **ftp** и пр.
 - Логин и пароль для аутентификации на сервере
 - Хост – доменное имя сервера **example.com**
 - Путь – путь к ресурсу на сервере
- Примеры URL
 - **http://example.com/index.html**
 - **/another/page.html**
 - **/page.html?firstName=John&lastName=Smith**
 - **https://ru.wikipedia.org/wiki/URL#Структура_URL**

Кодирование URL

- ⊗ Стандарт URL использует набор символов US-ASCII
 - Разрешается использовать лишь латинские буквы, цифры и несколько знаков пунктуации
 - Все другие символы необходимо перекодировать
- ⊗ Символы кириллицы и зарезервированные символы (/ , ? и пр.) преобразуются в UTF8
 - Б – Два байта D0 и 91
 - Б - %D0%91
- ⊗ Пример кодирования
 - <https://ru.wikipedia.org/wiki/Браузер>
 - <https://ru.wikipedia.org/wiki/%D0%91%D1%80%D0%B0%D1%83%D0%B7%D0%B5%D1%80>
- ⊗ Браузеры имеют ограничение по длине URL
 - Internet Explorer – 2048 символов

HTTP

- ⊙ HTTP (HyperText Transfer Protocol) — протокол прикладного уровня передачи данных
 - Текущая версия HTTP/1.1
 - Разработана финальная редакция черновика HTTP/2
- ⊙ Основой HTTP является технология «клиент-сервер»
 - Общение по схеме «запрос-ответ»
 - Промежуточное состояние не сохраняется
- ⊙ Клиент запрашивает ресурс, указывая URI в запросе
 - Ресурс HTML-страница, изображение, файл, форма и пр.
 - В запросе можно указать представление ресурса: кодировка, формат, язык и пр.
- ⊙ Сервер передает клиенту запрошенный ресурс в указанном формате
 - Ресурс может передаваться по частям

Сообщение HTTP

• Сообщение HTTP состоит из трёх частей:

- Стартовая строка — определяет тип сообщения
- Заголовки — устанавливают параметры передачи
- Тело сообщения — непосредственно данные сообщения

```
GET /index.html HTTP/1.1
Host: example.com
User-Agent: Mozilla/5.0
(X11; U; Linux i686; ru;
rv:1.9b5) Gecko/2008050509
Firefox/3.0b5
Accept: text/html
Connection: close
(пустая строка)
```

```
HTTP/1.1 200 OK
Date: Wed, 11 Feb 2009
11:20:59 GMT
Server: nginx/1.2.1
Last-Modified: Wed, 11 Feb
2009 11:20:59 GMT
Content-Language: ru
Content-Type: text/html;
charset=utf-8
Content-Length: 1234
Connection: close
(пустая строка)
(запрошенная страница HTML)
```

Методы HTTP

- ⊗ Метод GET - запрос содержимого указанного ресурса
 - Допускается передача параметров запроса через ?
 - Частичный GET – запрос фрагмента содержимого
 - Условный GET – содержимое передается только, если оно было изменено после указанной даты
 - Ответ сервера может кэшироваться

```
GET /path/resource?param1=value1&param2=value2 HTTP/1.1
```

- ⊗ HEAD - запрос метаданных
- ⊗ POST – передача пользовательских данных ресурсу
 - Ответ сервера не кэшируется
- ⊗ DELETE – удаляет указанный ресурс
- ⊗ TRACE - возвращает полученный запрос и информацию, добавленную или измененную промежуточными серверами

Разметка HTML, стили и скрипты

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Page Title</title>
```

<head> - секция
метаданных

```
  <style>
```

```
    <link rel="stylesheet" type="text/css" href="style.css">
```

```
  </style>
```

```
</head>
```

<body> - секция
отображаемой информации

```
<body>
```

```
  <script src="myScript.js"></script>
```

<script> - секция кода

```
  <p>My first paragraph.</p>
```

Содержимое страницы

```
</body>
```

```
</html>
```

Hyper Text Markup Language

- ⊙ HTML – стандартизированный язык разметки веб-документов
 - HTML5 – рекомендован к использованию с 2014 года
 - HTML 5.1 – рекомендован к использованию с 2016 года
- ⊙ Любой документ представляется в виде набора элементов, начало и конец которых обозначается специальными пометками — тегами
 - Теги определяют такие элементы, как заголовок, таблица, абзац и пр.
`<tagname> содержимое </tagname>`
 - Тег может иметь атрибуты
`<tagname attribute="value"> содержимое </tagname>`
- ⊙ Браузер отображает документ HTML учетом тегов
 - Chrome, IE, Firefox, Safari



Структура документа

```
<!DOCTYPE html>
```

Документ HTML5

```
<html>
```

Основной элемент

```
<head>
```

Метаданные страницы

```
<title>Page Title</title>
```

Название

```
</head>
```

```
<body>
```

Отображаемые данные

```
<h1>My First Heading</h1>
```

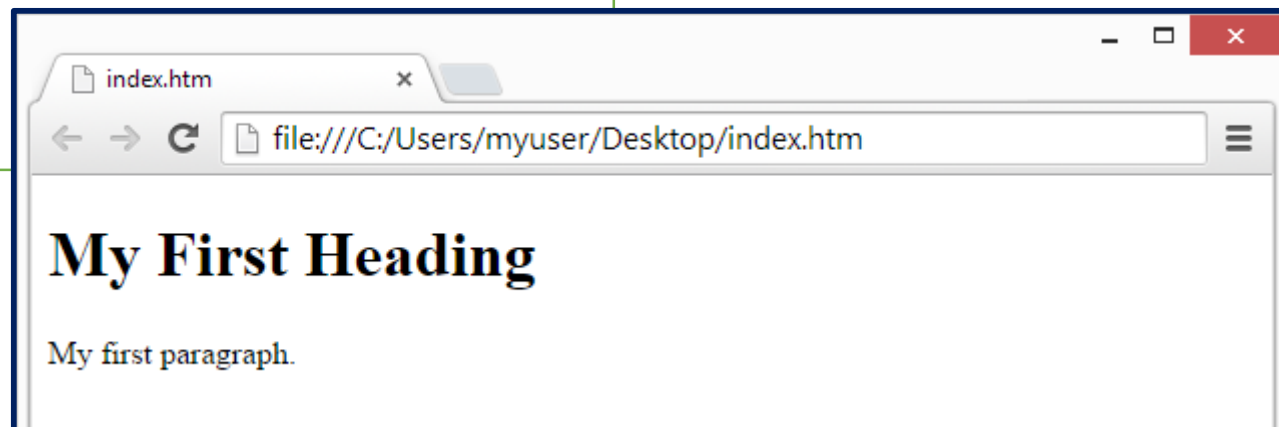
Заголовок

```
<p>My first paragraph.</p>
```

Абзац

```
</body>
```

```
</html>
```



Атрибуты тегов

- ⊙ Гиперссылка

```
<a href="http://qdots.ru">This is a link</a>
```

- ⊙ Медиа-файл

```

```

This is a link

- ⊙ Размер

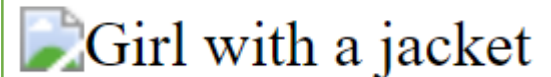
```

```

- ⊙ Альтернативный текст

```

```



- ⊙ Стил: цвет, шрифт, размер и пр.

```
<p style="color:red">I am a paragraph.</p>
```

I am a paragraph.

Заголовки

- ⦿ Заголовки описываются тегами от **<h1>** до **<h6>**
 - Службы поиска в первую очередь анализируют заголовки
 - Не следует использовать заголовки для форматирования обычного текста
- ⦿ Тег **<hr>** позволяет визуально разделять текст
- ⦿ Блок **<head>** содержит метаданные и не отображается

```
<head>
  <title> TestDoc </title>
</head>
<body>
  <h1>Heading 1</h1>
  <h2>Heading 2</h2>
  <h3>Heading 3</h3>
  <hr>
  <h4>Heading 4</h4>
  <h5>Heading 5</h5>
  <h6>Heading 6</h6>
```

Heading 1

Heading 2

Heading 3

Heading 4

Heading 5

Heading 6

Выделение текста

- ⊙ Полужирное выделение текста
 - `<b> Bold text </b>`
 - `<strong> Important text </strong>`
- ⊙ Курсив
 - `<i> Italic text </i>`
 - `<em> Emphasized text </em>`
- ⊙ Подчеркнутый текст
 - `<ins> Inserted text </ins>`
- ⊙ ~~Перечеркнутый~~ текст
 - `<del> Deleted text </del>`
- ⊙ Надпись ^{над} строкой и _{под} строкой
 - `<sup> Superscript text </sup>`
 - `<sub> Subscript text </sub>`

Абзацы

- Браузер автоматически удаляет лишние пробелы, пустые строки и пр. из документа HTML

This paragraph
contains a lot of
spaces
in the source code,
but the browser
ignores it.

This paragraph contains a lot of
spaces in the source code, but the
browser ignores it.

- Тег **<p> ... </p>** описывает новый параграф
- Тег **
** описывает переход на новую строку

`<p>This is
a
paragraph
with line
breaks</p>`

`<p>This is a new
paragraph</p>`

This is
a paragraph
with line breaks

This is a new paragraph

Формы

- Формы HTML описываются с помощью `<form> ... </form>`
 - Атрибут **action** описывает путь к скрипту обработки
 - Поле с вводом текста `<input type="text">`
 - После с выбором варианта `<input type="radio">`
 - Кнопка отправки `<input type="submit">`

```
<form action="/action_page.php">  
  Имя:<br>  
  <input type="text" name="firstname" value="Иван"><br>  
  Фамилия:<br>  
  <input type="text" name="lastname" value="Иванов"><br>  
  <br>  
  <input type="submit" value="Отправить">  
</form>
```

Имя:

Иван

Фамилия:

Иванов

Отправить

Элементы формы

☉ Поле с вводом текста

- `<input type="text">`
- `<input type="password">`

Пользователь:

Пароль:

☉ Кнопки

- `<input type="submit">` - отправляет данные на обработку
- `<input type="reset">` - сбрасывает данные в значение по умолчанию
- `<input type="button">` - может взаимодействовать с элементами на странице

☉ Поле с выбором варианта

- `<input type="radio">`
- `<input type="checkbox">`

☒ Мужчина

☐ Женщина

☐ Иное

Атрибуты полей ввода

```
<input type="text" name="firstname" value="John">
```

- **name** – обязательный параметр, иначе введенное в поле значение не будет обработано
- **value** – значение по умолчанию

⊕ Дополнительные атрибуты

- **readonly** – значение не изменяется
- **disabled** – значение не изменяется и не отправляется на обработку
- **size** – размер поля (по умолчанию 20)

⊕ Дополнительные атрибуты HTML5

- **autocomplete**
- **required**
- **list** – выпадающий список значений

Имя:

Фамилия:

disabled

Имя:

Иван

autocomplete

Имя:

 Заполните это поле.

required

Обработка формы

- Атрибут **action** определяет действие или скрипт, выполняемый при отправке формы

```
<form action="/action_page.php">
```

```
<form action="mailto:stud@qdts.ru">
```

- Как правило, это веб-страница на стороне сервера, содержащая скрипт, обрабатывающий данные
- Данные передаются в формате **firstname=John&lastname=Doe**

- Атрибут **target** определяет окно, в котором будет загружена итоговая веб-страница

```
<form action="/action_page.php" target="_blank">
```

- Значение по умолчанию **_self**
- **_blank** откроет новую страницу браузера

События элементов HTML

• Примеры событий

- Загрузка страницы
- Загрузка изображения
- Нажатие кнопки мыши, наведение мыши на элемент
- Изменение поля ввода
- Отправка формы и пр.

```
<form name="myForm" action="/action_page.php"
      onsubmit="return validateForm()" method="post">

<input type="text" id="number" onchange="validateInt()">

<input type="button" onclick="validate()" value="Check">
</form>
```

Пример функции проверки

```
<script>
function validateForm() {
    var x = document.forms["myForm"].number.value;
    if (isNaN(x)) {
        alert("Not a number");
        return false;
    }
}
</script>
<form name="myForm" action="/action_page.php"
    onsubmit="return validateForm()">

    Число: <input type="text" name="number">
    <input type="submit" value="Отправить">
</form>
```

Проверка введенных значений

⊗ Встроенная проверка

- Атрибуты **required**, **pattern**
- Тип поля ввода: **email**, **url**, **tel**, **number** и пр.
- Блокируют отправку формы до выполнения условий
- Может быть отключена с помощью атрибута формы **novalidate**
- Ограниченная поддержка в браузерах (HTML5)

⊗ Проверка с помощью JavaScript функций

- Функции, запускаемые по событию: изменению поля, нажатию кнопки, отправки формы и пр.
- Могут блокировать отправку формы

⊗ Проверка в скрипте-обработчике на стороне сервера

- «Последняя линия обороны» от неверных или вредоносных данных

Стили

Форматирование текста указывается через атрибут:

`<tagname style="property: value;">`

⊙ Цвет текста

`<p style="color:red;">This is a paragraph.</p>`

⊙ Цвет фона

`<body style="background-color:powderblue;">`

⊙ Шрифт

`<h1 style="font-family:verdana;">Heading</h1>`

⊙ Размер текста

`<h1 style="font-size:300%;">Heading</h1>`

⊙ Выравнивание

`<p style="text-align:center;">Centered</p>`

Внешние стили CSS

- CSS (Cascading Style Sheets) - формальный язык описания внешнего вида HTML-документа
 - Встроенное описание стиля `<p style="color:red;">`
 - Внутреннее описание: элемент `<style>`
 - Внешний файл CSS: элемент `<link>` в секции `<head>`

```
<!DOCTYPE html>
<html>
<head>
  <link rel="stylesheet"
        href="styles.css">
</head>
<body>

  <h1>This is a heading</h1>
  <p>This is a
      paragraph.</p>
</body>
</html>
```



```
body {
  background-color: #6a5acd;
}
h1 {
  color: blue;
}
p {
  color: red;
}
```

Что такое CSS?

CSS (*Cascading Style Sheets* — каскадные таблицы стилей) — формальный язык описания внешнего вида документа, написанного с использованием языка разметки

- ⊙ Разделение описания логической структуры веб-страницы (HTML или другие языки разметки) от описания внешнего вида этой веб-страницы (CSS)
- ⊙ Увеличение доступности документа, предоставление большей гибкости и возможности управления его представлением
- ⊙ Уменьшение сложности и повторяемости в структурном содержимом
- ⊙ Возможность представлять один и тот же документ в различных стилях или методах вывода (экранное представление, печатное представление, чтение голосом, шрифт Брайля и пр.)

Подключение CSS к документу

- ⊛ Правила CSS пишутся на формальном языке CSS и располагаются в таблицах стилей
- ⊛ Таблицы стилей могут располагаться в:
 - Самом веб-документе
 - Внешних файлах, имеющих формат CSS
 - С помощью тега **link**
 - С помощью директивы **@import**
- ⊛ Выбор метода подключения влияет на зону действия таблицы стилей

Описание CSS внутри элемента

- ⊗ Может располагаться в описание отдельного тега
 - С помощью атрибута **style**
- ⊗ Правила действуют только на содержимое этого тега
- ⊗ Пример описания стилей элемента

```
<!DOCTYPE html>
<html>
  <head>
    . . . . .
  </head>
  <body>
    <p style="font-size: 20px; color: green; font-
family: arial, helvetica, sans-serif">
      . . . . .
    </p>
  </body>
</html>
```

Описание CSS внутри документа

- Может располагаться между тегами `<style>` и `</style>`
 - Внутри тегов `<head>` и `</head>`
- Правила таблицы действуют на протяжении всего документа
- Пример внутренней таблицы стилей

```
<!DOCTYPE html>
<html>
  <head>
    .....
    <style>
      body {
        color: red;
      }
    </style>
  </head>
  <body>
    .....
  </body>
</html>
```

Подключение внешнего CSS к документу

- ⊙ Может быть подключена с помощью тега `<link>`
 - Внутри тегов `<head>` и `</head>`
- ⊙ Атрибут **href** указывает адрес таблицы стилей
- ⊙ Правила таблицы стилей действуют на протяжении всего документа
- ⊙ Пример внешней таблицы стилей

```
<!DOCTYPE html>
<html>
  <head>
    .....
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    .....
  </body>
</html>
```

Подключение внешнего CSS к документу

- ⊙ Может быть подключена с помощью директивы **@import**
 - Директива **@import** располагается сразу после тега **<style>**
 - Директива **@import** указывает адрес таблицы стилей через ключевое слово **url**
- ⊙ Правила таблицы действуют на протяжении всего документа
- ⊙ Пример подключения внешней таблицы стилей

```
<head>
...
    <style media="all">
        @import url(style.css);
    </style>
</head>
```


Правила построения CSS

- ⊗ Каждое правило CSS из таблицы стилей имеет две основные части
 - Селектор
 - Блок объявлений
- ⊗ Селектор определяет, на какие части распространяется правило
- ⊗ Блок объявлений состоит из одного или более объявлений
- ⊗ Объявление – это сочетание свойства CSS и значения

```
селектор, селектор {  
    свойство: значение;  
    свойство: значение;  
    свойство: значение;  
}
```

Виды селекторов

- Универсальный селектор (*)
- Селектор тегов (p)
- Селектор классов (.note)
- Селектор идентификаторов (#paragraph1)
- Селектор атрибутов (a[href])
- Селектор потомков (div#paragraph1 p.note)
- Селектор дочерних элементов (p.note > b)
- Селектор сестринских элементов (h1 + p)
- Селектор псевдоклассов (a:active)
- Селектор псевдоэлементов (p::first-letter)

Селекторы классов и идентификаторов

- Класс или идентификатор может быть присвоен элементу HTML посредством атрибутов **class** или **id**
- Идентификатор предназначен для одного элемента
- Идентификаторы широко используются в JavaScript для нахождения уникального элемента в документе

```
#paragraph1 {  
    margin: 0;  
}
```

- Класс обычно присваивают сразу нескольким элементам

```
.note {  
    color: red;  
    background-color: yellow;  
    font-weight: bold;  
}
```

Наследование стилей

- ⦿ Свойства CSS, объявленные для элементов-предков, наследуются элементами потомками
- ⦿ Наследование идет до тех пор, пока наследуемое свойство не будет в явном виде указано для элемента потомка: далее запускается новая цепочка наследования
- ⦿ Не все свойства CSS наследуются (работает для менее тривиальных задач: рамка не наследуется, цвет наследуется)

Каскадирование стилей

Применяется, когда элементу одновременно поставлено в соответствие более одного правила CSS:

- ⊙ Наиболее низким приоритетом обладает стиль браузера
- ⊙ Следующим по значимости является стиль, заданный пользователем браузера в его настройках
- ⊙ Наиболее высоким приоритетом обладает стиль, заданный непосредственно автором страницы
- ⊙ В авторском стиле приоритеты расставляются своим образом
- ⊙ Более высоким приоритетом обладают стили, объявленные непосредственно в теге элемента с помощью атрибута `style`
- ⊙ Самым высоким приоритетом обладают стили, объявленные автором страницы или пользователем, с помощью сопроводительного слова `!important`

Приоритеты в авторском стиле

- ⦿ Самым низким приоритетом обладают стили, наследуемые в документе элементом от своих предков
- ⦿ Более высоким приоритетом обладают стили, заданные во внешних таблицах стилей, подключённых к документу
- ⦿ Ещё более высоким приоритетом обладают стили, заданные непосредственно селекторами всех десяти видов
- ⦿ Если к элементу имеют отношение несколько селекторов, то конфликты разрешаются с помощью расчёта специфичности каждого селектора
- ⦿ Селекторы применяются в порядке убывания специфичности
 - $(1,0,0)$ за каждый объявленный идентификатор в селекторе правила CSS
 - $(0,1,0)$ за каждый объявленный класс, атрибут и псевдокласс в селекторе правила CSS
 - $(0,0,1)$ за каждый объявленный элемент и псевдоэлемент в селекторе правила CSS)
 - $(1,0,0) > (0,100,0)$ (имеет большую специфичность, т.е. больший приоритет)
 - $(0,1,0) > (0,0,100)$
 - Если специфичности одинаковые, то применяется селектор, описанный ниже в документе

CSS свойства

- ⊗ Цвет элемента: **color**
 - HTML поддерживает 140 стандартных имен и модель RGB
- ⊗ Задний фон: **background-color**, **background-image**, **background-repeat**, **background-attachment** и пр.
- ⊗ Границы элемента: **border-style**, **border-width**, **border-color**, **border-radius**, **border-individual-style**
- ⊗ Размер элемента: **height**, **max-height**, **min-height**, **width**, **max-width**, **min-width**
- ⊗ Поле вокруг элемента за границами элемента (отступ): **margin-top**, **margin-right** и пр.
- ⊗ Поле элемента внутри границ: **padding-top** и пр.
- ⊗ Поле за пределами внешней границы элемента: **outline-color**, **outline-offset**, **outline-style** и пр.
- ⊗ Параметры размещения текста внутри элемента: **direction**, **letter-spacing**, **line-height**, **text-align**, **text-decoration**, **text-indent**, **text-shadow** и пр.

Примеры свойств CSS

```
<!DOCTYPE html>
<html>
<head>
<style>
    div {
        margin: 70px;
        border: 1px solid #4CAF50;
    }
</style>
</head>
<body>

<div>У этого элемента отступ 70px.</div>

</body>
</html>
```

У этого элемента отступ 70px.

Примеры свойств CSS

```
<head>
<style>
    div {
        border: 1px solid black;
        background-color: lightblue;
        padding-top: 50px;
        padding-right: 30px;
        padding-bottom: 50px;
        padding-left: 80px;
    }
</style>
</head>
<body>
    <div>У этого элемента верхнее поле 50px, правое поле 30px,
    нижнее поле 50px и левое поле 80px.</div>
</body>
```

У этого элемента верхнее поле 50px, правое поле 30px, нижнее поле 50px и левое поле 80px.

Уровни CSS

- ⊗ CSS1 – 17 декабря 1996 г.
 - Параметры шрифтов, цвета, атрибуты текста, выравнивание для текста и других элементов, свойства блоков
- ⊗ CSS2 – 12 мая 1998 г.
 - Блочная верстка, типы носителей, звуковые таблицы стилей, страничные носители, расширенный механизм селекторов, указатели, генерируемое содержимое
- ⊗ CSS2.1 – 7 июня 2011 г.
 - Исправление ошибок, некоторые части спецификации изменены или удалены
- ⊗ CSS3 – в процессе разработки
 - Анимированные элементы без JS, сглаживание, тени, поддержка радиальных и линейных градиентов и пр.
- ⊗ CSS4 – в процессе разработки (черновик)

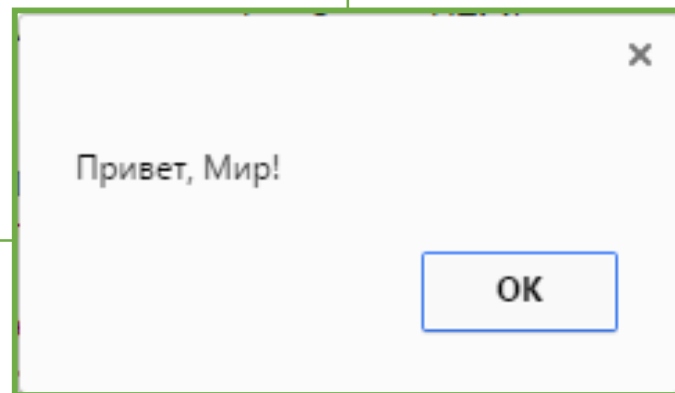
Язык JavaScript

- ⊗ JavaScript – это язык программирования, широко применяемый для создания интерактивных сайтов
 - Реакция сайта на нажатие кнопки, на ввод данных и пр.
 - Использование динамических стилей
 - Анимация
 - Игры
 - Передача данных на сервер (AJAX)
- ⊗ JavaScript полностью интегрирован HTML/CSS
 - Возможно изменение любых объектов HTML из скрипта
 - Возможно изменение любых стилей из скрипта
- ⊗ JavaScript поддерживается всеми браузерами
- ⊗ Спецификация ECMAScript описывает синтаксис, базовые объекты и алгоритмы JavaScript

Встроенные скрипты. Тег <script>

```
<!DOCTYPE HTML>
<html>
<head>
  <meta charset="utf-8">
</head>

<body>
  <p>Начало документа...</p>
  <script>
    alert( 'Привет, Мир!' );
  </script>
  <p>Конец документа</p>
</body>
</html>
```



Внешние скрипты

```
<script src="/path/to/script.js"></script>
```

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/lodash.js/4.3.0/lodash.js"></script>
```

- ⦿ **ВНИМАНИЕ:** В одном теге `<script>` нельзя одновременно подключить внешний скрипт и указать код.

```
<script src="file.js">  
// Не будет выполнено  
    alert(1);  
</script>
```

```
<script src="file.js">  
</script>  
<script>  
    alert( 1 );  
</script>
```

Асинхронный запуск скрипта

```
<html>
<head>
    <script
      src="big.js">
    </script>
</head>

<body>
    Этот текст не будет
    показан, пока
    браузер не выполнит
    big.js.
</body>
</html>
```

⊕ Атрибут **async**

`<script async src=...>`

- Скрипт будет загружен асинхронно, как только загрузится в кэш

⊕ Атрибут **defer**

`<script defer src=...>`

- Скрипт будет запущен асинхронно после того, как документ будет загружен в кэш
- Гарантируется порядок выполнения скриптов

Синтаксис JavaScript

• Команды разделяются точкой с запятой (;)

- Необязательно, но рекомендовано

```
<script>
    alert( 'Привет, Мир!' );
    alert( 'Привет, друг!' );
</script>
```

• Комментарий

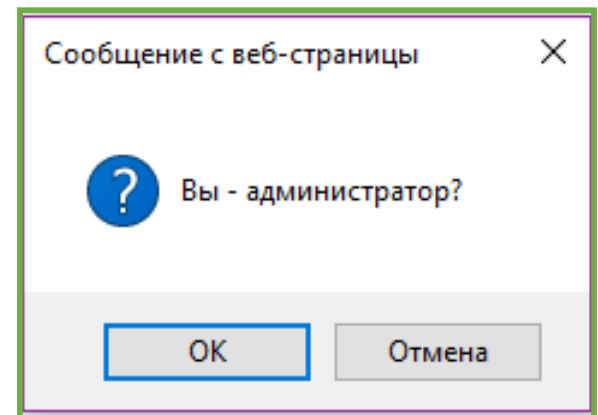
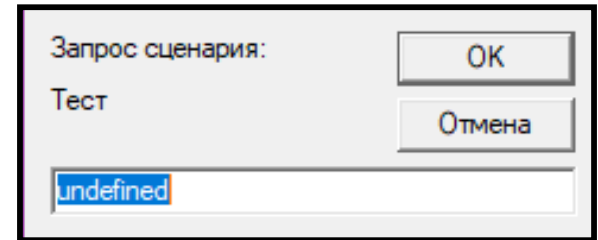
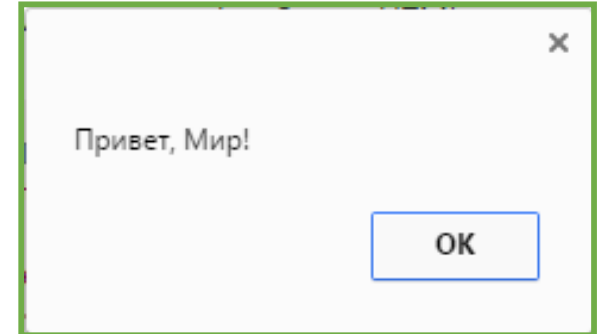
- Ctrl + / - установить или убрать комментарий

```
<script>
    // Это однострочный комментарий

    /* Длинный комментарий
       в несколько строк
    */
</script>
```

Взаимодействие с пользователем

- Встроенные функции взаимодействия
 - Скрипт останавливается, пока не будет получен ответ от пользователя
 - Внешний вид и расположение окон определяет браузер
- **alert(сообщение)**
- **result = prompt(title, default);**
 - При нажатии CANCEL или ESC, результат null или `` (Safari 5.1)
 - Значение по умолчанию undefined. Желательно переопределить
- **result = confirm(question);**
 - true, если нажат OK
 - false, если нажат CANCEL



Итоги

В данной лекции были рассмотрены следующие темы:

-  Протокол HTTP
-  Технологии WWW
-  Язык разметки HTML
-  Каскадные стили CSS
-  Интерактивность веб-страниц: JavaScript